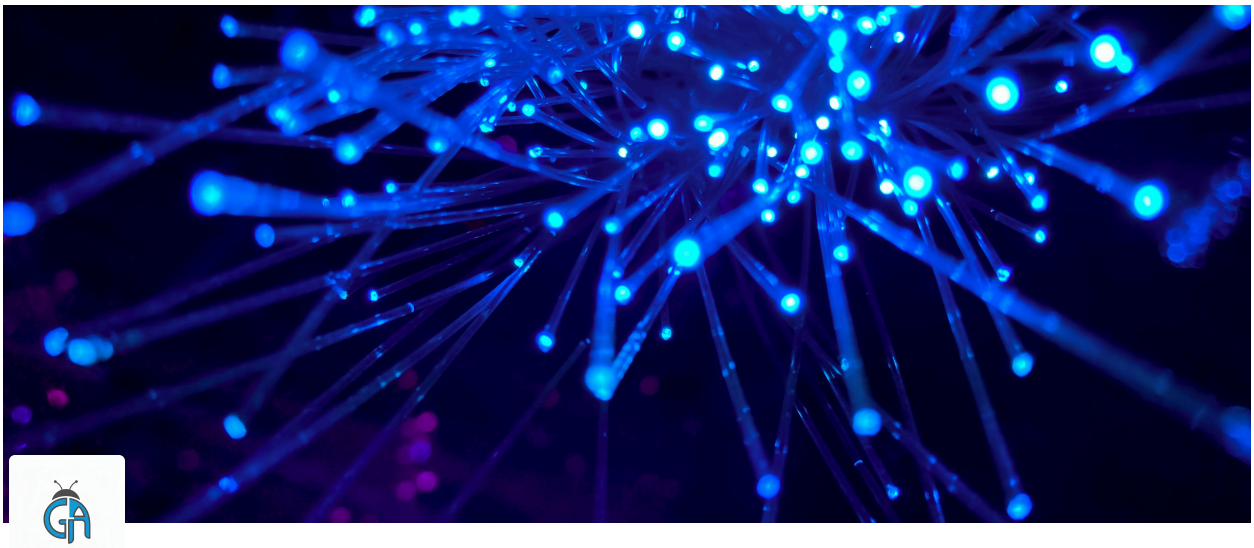

📊 Decision Table Testing

Simplifying Complex Logic

🏷️ Tags: QA, Test Design, Techniques

⏱️ Reading Time: 3 min

🚀 TL;DR

When the rules are complicated (e.g., "If User is New AND has a Coupon, but NOT on a Weekend..."), use a **Decision Table**.

It maps every possible combination of inputs to the correct output.

🧐 The Core Concept

Sometimes, an application's behavior depends on a combination of many different inputs. Writing test cases for this can get messy.

A Decision Table (also called a "Truth Table") organizes this logic into a simple grid.

- **Conditions:** The Inputs (Questions we ask).
- **Actions:** The Outputs (What the system should do).
- **Rules:** The specific combinations (The columns).

☕ The Barista Analogy

Imagine a coffee shop has complex pricing rules.

- **Rule 1:** If you bring your own cup, you get $0.50 off.
- **Rule 2:** If you are a member, you get $1.00 off.
- **Rule 3:** If you do both, you get $2.00 off.

If you just guess, you might miss a case. A **Decision Table** lists every customer type (Member/Non-Member) vs. every Cup type (Own/Paper) to ensure the cashier never makes a mistake.

💡 Real World Example: E-Commerce Discount

The Logic:

1. Are you a **New User**?
2. Do you have a **Coupon Code**?

The Rules:

- New User + Coupon = **20% Discount**
- New User + No Coupon = **10% Discount**
- Old User + Coupon = **5% Discount**
- Old User + No Coupon = **0% Discount**

The Decision Table:

Condition / Rule	Rule 1	Rule 2	Rule 3	Rule 4

Is New User?	Yes	Yes	No	No
Has Coupon?	Yes	No	Yes	No
ACTION: Discount	20%	10%	5%	0%

🧠 Why This Matters (The "Impossible" Bug)

Without a Decision Table, testers often test **Rule 1** (The Happy Path) and **Rule 4** (The Negative Path), but they forget **Rule 2 and 3**.

Using this technique guarantees **100% Logic Coverage**. It gives you the confidence to say: *"I have tested every mathematically possible combination."*